

Link State Routing

Link State Routing (LSR) is a routing algorithm in which **each router builds a complete map of the entire network topology** and independently calculates the shortest path to every other router using **Dijkstra's algorithm**.

It is used in real-world routing protocols like **OSPF** (Open Shortest Path First) and **IS-IS** (Intermediate System to Intermediate System).

Basic Idea

Instead of sharing full routing tables with neighbors (as in Distance Vector Routing), each router:

1. Learns its directly connected neighbors
2. Measures the cost of each link
3. Creates a **Link State Packet (LSP)**
4. Floods this information to all routers
5. Builds a complete network graph
6. Runs **Dijkstra's Shortest Path Algorithm**
7. Constructs its routing table

Why Link State Routing is Needed?

Distance Vector Routing (like **RIP**) had problems:

- Count-to-infinity problem
- Routing loops
- Slow convergence
- Limited scalability

Link State Routing solves these by giving every router a **global view** of the network.

Working of Link State Routing

1. Neighbor Discovery

Each router identifies its directly connected neighbors.

2. Link Cost Measurement

Cost may be based on:

- Bandwidth
- Delay
- Load
- Administrator-defined metric

3. Create Link State Packet (LSP)

Contains:

- Router ID
- List of neighbors
- Cost to each neighbor
- Sequence number
- TTL

4. Flooding

LSPs are sent to **all routers** in the network.

5. Build Link State Database (LSDB)

Each router builds the same topology map.

6. Run Dijkstra Algorithm

Each router computes the **Shortest Path Tree (SPT)** from itself.

Feature	Link State Routing
Network Knowledge	Complete topology
Algorithm Used	Dijkstra
Convergence	Fast

Loop Problem	Very rare
Scalability	High
Example Protocol	OSPF

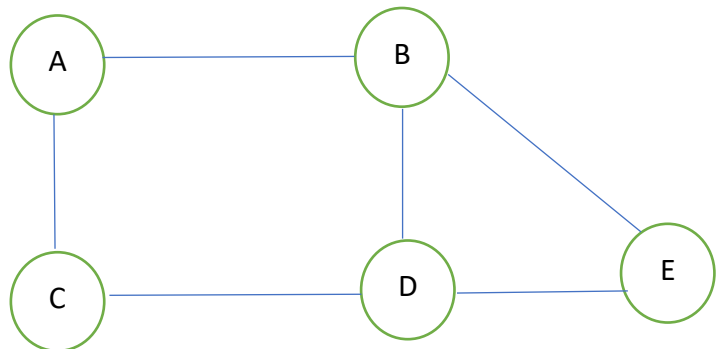
Advantages

- Fast convergence
- No count-to-infinity problem
- Loop-free paths
- Suitable for large networks

Disadvantages

- Requires more memory
- Higher CPU usage
- More complex implementation

Given Network



Link Costs:

- A-B = 3
- A-C = 2
- C-D = 1
- B-D = 4
- B-E = 6
- D-E = 2

Compute shortest paths from **Router A**

SOLUTION

Step 1: Initialization

Node	Distance from A	Previous
A	0	—
B	∞	—

C	∞	—
D	∞	—
E	∞	—

Visited Set $S = \{ \}$

Step 2: Add A to S

Update neighbors of A:

- $B = 3$
- $C = 2$

Node	Distance	Previous
A	0	—
B	3	A
C	2	A
D	∞	—
E	∞	—

$S = \{A\}$

Step 3: Choose Minimum ($C = 2$)

Add C to S.

Update neighbors of C:

- $D \text{ via } C = 2 + 1 = 3$

Node	Distance	Previous
A	0	—
B	3	A
C	2	A
D	3	C
E	∞	—

$S = \{A, C\}$

Step 4: Choose Minimum (B or D = 3)

Choose **B** first.

Update neighbors of B:

- D via B = $3 + 4 = 7$ (not better than 3)
- E via B = $3 + 6 = 9$

Node	Distance	Previous
A	0	—
B	3	A
C	2	A
D	3	C
E	9	B

$S = \{A, C, B\}$

Step 5: Choose Minimum (D = 3)

Add D to S.

Update neighbors of D:

- E via D = $3 + 2 = 5$ (better than 9 ✓)

Node	Distance	Previous
A	0	—
B	3	A
C	2	A
D	3	C
E	5	D

$S = \{A, C, B, D\}$

Step 6: Choose Minimum (E = 5)

Add E to S.

No further updates.

$S = \{A, C, B, D, E\}$

Algorithm ends.

Final Shortest Paths from A

Destination	Cost	Path
B	3	$A \rightarrow B$
C	2	$A \rightarrow C$
D	3	$A \rightarrow C \rightarrow D$
E	5	$A \rightarrow C \rightarrow D \rightarrow E$

Final Routing Table at A

Destination	Next Hop	Cost
B	B	3
C	C	2
D	C	3
E	C	5

Comparison : DVR vs LSR

Distance Vector Routing (DVR)	Link State Routing (LSR)
Shares routing table with neighbors	Shares link-state information with all routers
Knows only neighbors	Knows complete network topology
Uses Bellman-Ford algorithm	Uses Dijkstra algorithm
Periodic updates	Event-driven updates
Slow convergence	Fast convergence
Count-to-infinity problem exists	No count-to-infinity problem
Routing loops possible	Very rare routing loops
Low memory requirement	High memory requirement
Low CPU usage	Higher CPU usage
Suitable for small networks	Suitable for large networks
Metric usually hop count	Supports multiple metrics (bandwidth, delay, cost)
Example: RIP	Example: OSPF, IS-IS

Practice Question on Link State Routing

Q1. Consider following Routers: A, B, C, D, E

1. Apply Link State Routing using Dijkstra's Algorithm at router A
2. Construct the final routing table at A and find the shortest path from A to E.

Link Costs are given as follows:

From	To	Cost
A	B	3
A	C	1
B	C	7
B	D	5
C	D	2
D	E	4
C	E	8